Operating Systems and Parallel Computing

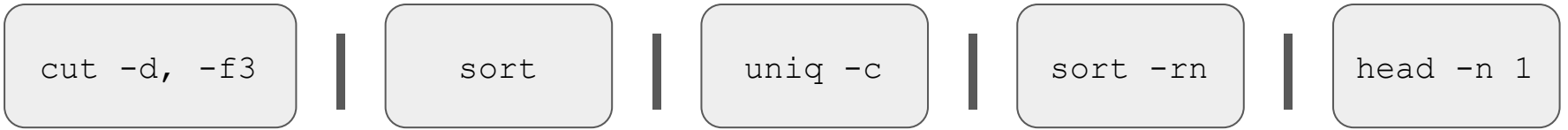
Lecture 3

Prof. Matteo Turilli

Concatenating Commands

- The pipe command '|' is used to concatenate commands
- Concatenation: the output of a command becomes the input of another command
- Unix philosophy: many extremely fast and efficient, capability-specific tools (i.e., the commands) that can be arbitrarily combined to create complex capabilities.
- Problem: given our `sales_2025_sample.csv` dataset, what is the region with most records?
- Solution based on the commands we have studied:
  a. (Optional) skip the headers: `tail -n +2 sales_2025_sample.csv`
  b. Extracts the `region` column: `cut -d ',' -f 3`
  c. Groups identical regions together: `sort`
  d. Counts records for each region: `uniq -c`
  e. Sorts numerically, largest count first: `sort -nr`
  f. Returns the count and region with the most records: `head -n 1`
- Question: Which command do we use and concatenate for each step a–f?

Concatenating Commands



- Each tool does a specific job
- Pipes feed each output into the next command
- Note: no temporary files, no spreadsheet, no mouse overhead
- Fast! 0.051s for 100K rows

```
analyst@5705313fee66:~/data$ time cut -d, -f3 sales_2025_sample.csv | sort | uniq -c | sort -rn | head -1
  25928 Iberia

real    0m0.051s
user    0m0.047s
sys     0m0.014s
```

Pipeline Best Practices

- Conceptualize the solution in terms of a pipeline, thinking about the commands you know and imagining their individual results before a concatenation.
- Explore the commands capabilities reading the command man page.
- Once we ‘see’ the solution, we build the pipeline one piece at a time.
- Add one stage, run it, evaluate the output, then add the next once satisfied:
  - Run cut: check that is the right column and note that there are groups of regions.
  - Add | sort: verify that the command grouped the column.
  - Add | uniq -c: verify that the count flag worked as expected.
  - Add | head -1: this returns the answer to the given problem
- If something looks wrong, we know exactly which stage broke it.
- A long command that came out of nowhere is difficult to conceptualize (it becomes easier with time) but impossible to debug.
- Growing the pipeline is equivalent to prototype and test each stage before running the whole one. If performance is a problem, we use head to use a subset.

Pipeline Best Practices

- In most operating systems, each command has associated streams, i.e., communication channel between a computer program and its environment.
- Three standard (i.e., default) streams:
  - Standard Input (`stdin`): what data (bytes) goes into the command
  - Standard Output (`stdout`): what data the command returns upon successful execution
  - Standard Error (`stderr`): what data the command returns upon encountering an error



This is an interactive class

- Please run our course container and try out all the commands in the next slides
- Run Docker Desktop and then start our container

```
docker compose run --rm ospc
```

Standard Stream Redirection

- In Unix-like systems, “everything is a file” so we can have diverse data sources and targets, as far as they can be represented as producer/consumers of bytes.
- When we use `A|B` we redirect the stdout of A to stdin of B
- We use specific operators to redirect each individual stream (stdout is the default)
- `>` : redirect a program’s stdout
  - `sort db.csv > sorted_db.csv` (WARNING: overwrite `sorted_db.csv`)
- `<` : redirect stdin to a program
  - `sort < data.csv`
  - `grep -i "error" < app.log`
- `>>` : redirect stdout appending its content to the target
  - `sort db.csv >> sorted_db.csv` (NOTE: does not overwrite `sorted_db.csv`)
- `2>` : redirect stderr
  - `find / -name x 2> errors.txt` (STD streams are numbered: 0 = input, 1 = output, 2 = error)
  - `your_command > all.txt 2>&1` (merges STDERR and STDOUT)

Pipe and Redirect

- Use a pipe | when the next step is another command.
- Use a redirect > when the next step is a file on disk.
- They combine naturally: a pipeline does the work, and > saves the final result.

```
analyst@5705313fee66:~/data$ time cut -d, -f3 sales_2025_sample.csv | sort | uniq -c | sort -rn > ~/work/sorted_region_sales.txt

real    0m0.207s
user    0m0.138s
sys     0m0.121s
analyst@5705313fee66:~/data$ cat ~/work/sorted_region_sales.txt
  25928 Iberia
  22153 DACH
  15975 France
  13962 Italy
  12023 Benelux
   9959 Nordics
      1 region
```

- Why do we have ‘1 region’ at the end of the file?
- Can we eliminate it?

Performance Analysis

- Writing to disk takes time, even with modern disks, at least until cached
- Compare the time it takes to write the output to a file

```
analyst@5705313fee66:~/data$ time cut -d, -f3 sales_2025_sample.csv | sort | uniq -c | sort -rn > ~/work/sorted_region_sales.txt

real    0m0.207s
user    0m0.138s
sys     0m0.121s
```

- Or to the terminal

```
analyst@5705313fee66:~/data$ time cut -d, -f3 sales_2025_sample.csv | sort | uniq -c | sort -rn
  25928 Iberia
  22153 DACH
  15975 France
  13962 Italy
  12023 Benelux
   9959 Nordics
      1 region

real    0m0.101s
user    0m0.066s
sys     0m0.060s
```

- Are these measures statistically significant?

Exercise Time

- Now that we know the bases, let’s have some fun!
- Problem: What are the top three selling categories for the Iberian region?
- Remember:
  - Build the pipeline one step at a time
  - For each step, first think about what result you expect and then test it
  - Once a step works as intended move to the next one
  - Once the full pipeline is written and tested, write the results in ~/work so that you do not lose it when you reboot the course container.

Exercise Time

- Now that we know the bases, let’s have some fun!
- Problem: What are the top three selling categories for the Iberian region?
- Remember:
  - Build the pipeline one step at a time
  - For each step, first think about what result you expect and then test it
  - Once a step works as intended move to the next one
  - Once the full pipeline is written and tested, write the results in ~/work so that you do not lose it when you reboot the course container.

```
analyst@5705313fee66:~/data$ grep 'Iberia' sales_2025_sample.csv | cut -d, -f4 \
> | sort | uniq -c | sort -rn | head -3
   4400 Sports
   4398 Apparel
   4331 Grocery
analyst@5705313fee66:~/data$ grep 'Iberia' sales_2025_sample.csv | cut -d, -f4 \
> | sort | uniq -c | sort -rn | head -3 > ~/work/iberia_top.txt
```

From Commands to Scripts

- Our example pipeline implements a pattern: for a given dataset and region, it extracts the top n selling categories.
- We can write a pipeline for each dataset, region, and value of n, or we can save a generalized version of our pipeline in a reusable script.
- Behind the prompt (i.e., where we input the commands) we have been using there is a programmable runtime environment called ‘shell’. Specifically, we have been using the `bash` shell.
- In this environment, we can define variables, loops, if/else statements, etc. Thus, for each shell we have a programming language with which we can write scripts.
- A complete overview of the bash programming language is not useful for this course
- We look at how to define variables and use them to generalize our example pipeline.

From Commands to Scripts

- We `cd ~/work` and we use `vi summarize.sh` to edit our script:

```
#!/bin/bash

DATASET=$1
REGION=$2
NUMBER=$3

grep "$REGION" "$DATASET" | cut -d, -f4 | sort | uniq -c | sort -rn | head -n "$NUMBER"
```

- $n are positional arguments and we assign them to meaningful variables that we then use in the pipeline.
- We save and quit with Esc :wq
- We make the file executable: `chmod +x summarize.sh` (more on this later)
- And execute with, e.g.,
  - `./summarize.sh ~/data/sales_2025_sample.csv Iberia 3`
- NOTE: we need to respect the order of our arguments
- Our script is a new command we can pipe and redirect as needed

Shell and Python Work Together

- We don’t have to choose between the shell and Python. They call each other.
- From Python:
  - `subprocess.run([...])` runs a shell command and hands back its output
- From the shell:
  - `python3 my_script.py` runs the Python interpreter as a command
- In this way, we can use each for what it’s best at: the shell to move and filter data, Python to compute on it.
- `cd work; vi ptos.py`:
- We write a Python program that uses our `summarize.sh` to get the top five selling categories (shell does the data processing) and then sums the total sold items (python does the calculation).

Shell and Python Work Together

- We pass some of the positional arguments from Python to the shell
- We sun the shell in a subprocess and retrieve the returned object `subprocess.CompletedProcess`
- We access the `stdout` attribute and sum the counts of each category
- We print the results to the terminal and we exit
- How do we know about subprocess?
- We need to read the documentation at https://docs.python.org/3/library/subprocess.html after the next two lectures!

```python
import subprocess
import sys

dataset = sys.argv[1]
region = sys.argv[2]

result = subprocess.run(
    ["bash", "summarize.sh", dataset, region, "5"],
    capture_output=True,
    text=True,
    check=True
)

total = 0

for line in result.stdout.splitlines():
    count, category = line.split(maxsplit=1)
    total += int(count)

print("Output from summarize.sh:")
print(result.stdout)

print("Sum of the top 5 category counts:", total)
```

Exercises and Homework

- Working in the container, build each answer as a pipeline, then save it to `~/work`.
- Which product category is most common overall?
  - Hint cut the category column, then group/count/rank
- How many orders came from the “store” channel?
  - Save the number to `~/work/store_count.txt`.
- For the DACH region only, list the top 3 categories.
  - Hint: grep the region first, then pipe or check how to do sorting on multiple columns
- Turn your DACH pipeline into a script `region_top.sh` that takes the region name as `$1`.
- Practise building pipelines from the cheat sheet; fluency here pays off all semester.
- Save anything you want to keep into `~/work` (everything else resets on restart).
- Next time: we will deepdive into processes, i.e., what actually happens when you run a command, and how to watch it with `ps`, `top`, and `htop`.
- Read Chapter 4 of “Operating Systems: Three Easy Pieces”